

# GPTQ: 대규모 언어 모델의 정확한 사후 양자화

GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers

---

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, Dan Alistarh (2023)

재학습 없이 weight를 줄이되, calibration 입력에서 layer 출력을 보존하는 방법

# 왜 weight memory가 출발점인가

175B 규모에서는 추론을 시작하기 전부터 weight 저장에 여러 GPU를 요구한다

| 대상         | 논문의 값         | 의미                         |
|------------|---------------|----------------------------|
| GPT-3 175B | FP16 약 326 GB | weight만으로 단일 고용량 GPU를 넘음   |
| 기존 선택      | 다중 GPU 추론     | 비용과 배치 복잡도가 증가             |
| GPTQ의 질문   | 3비트 또는 4비트    | 재학습 없이 layer 출력을 보존할 수 있는가 |

---

압축률만이 아니라 양자화 뒤의 출력 오차가 핵심 제약이다

---

# 먼저 구분할 네 가지 용어

| 용어               | 뜻                  | GPTQ에서의 역할               |
|------------------|--------------------|--------------------------|
| Weight-only      | weight만 낮은 bit로 표현 | activation 양자화는 실험 범위 밖  |
| PTQ              | 학습 뒤에 수행하는 양자화     | 재학습이나 fine-tuning을 하지 않음 |
| Calibration      | 작은 대표 입력 표본        | layer 입력 X와 민감도를 추정      |
| Inference kernel | 압축 weight로 곱셈을 실행  | GPTQ의 값 선택 규칙과 별도 계층     |

---

GPTQ는 weight-only one-shot PTQ이며 새 Transformer 구조가 아니다

---

# 양자화 grid는 허용된 값의 집합이다

quant(w)는 full-precision weight를 정해진 grid 값으로 보낸다

| 구성          | 예시                  | 주의할 점                  |
|-------------|---------------------|------------------------|
| Full weight | 0.80, 0.55          | 아직 연속적인 값              |
| 교육용 grid    | {0, 1}              | 실제 논문 설정보다 단순한 예시      |
| Weight code | 0 또는 1              | 저장되는 양자화 level의 번호     |
| Scale와 zero | grid를 복원하는 metadata | code bit 수와 전체 저장량은 다름 |

---

논문의 주 실험은 row별 비대칭 min-max grid를 사용한다

---

# 독립 반올림 전에 행렬 기호부터 읽자

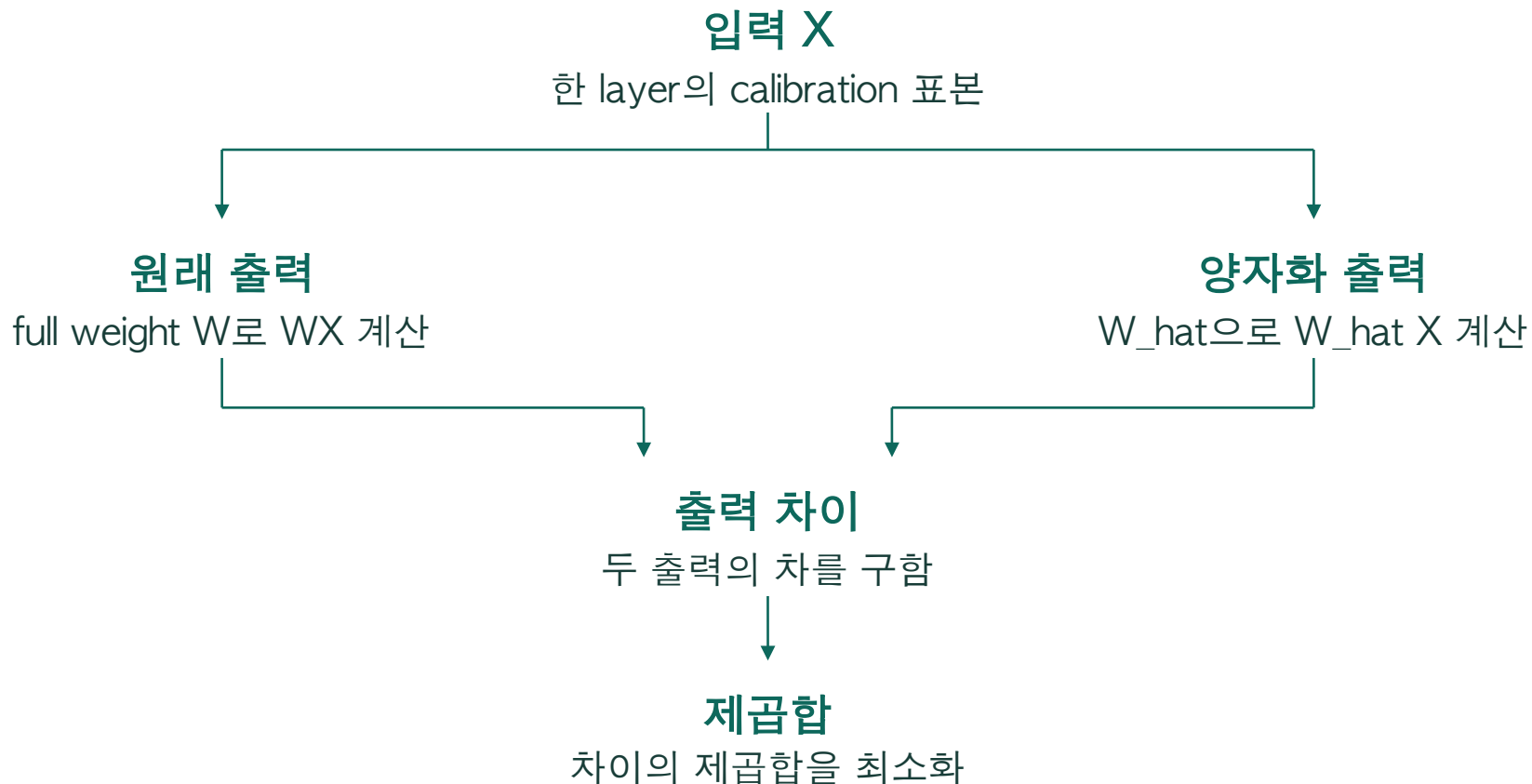
한 linear layer의 출력은 weight와 입력의 곱으로 결정된다

| 기호       | 교육용 값           | 뜻                             |
|----------|-----------------|-------------------------------|
| $w$      | $[0.80, 0.55]$  | $W$ 의 한 row                   |
| $X$      | $[[1,0],[1,1]]$ | 두 calibration 입력을 column으로 배치 |
| $wX$     | $[1.35, 0.55]$  | 원래 row의 두 출력                  |
| $W\_hat$ | 양자화된 $W$        | 허용된 grid 값으로 만든 행렬            |

작은 weight 차이보다 calibration 입력에서의 출력 차이를 본다

# GPTQ의 목표는 layer 출력 재구성이다

목표: calibration 입력  $X$ 에서  $\|WX - \hat{W}X\|^2$ 를 작게 만든다



# Hessian은 입력 방향의 민감도를 담는다

초기 H와 iteration의  $H_F$ 를 구분하면 OBQ 보상식을 정확히 읽을 수 있다

| 기호         | 교육용 값                | 읽는 방법                    |
|------------|----------------------|--------------------------|
| H          | $2XX^T$              | 양자화 시작 전 전체 좌표의 Hessian  |
| $H_F$      | $2X_F X_F^T$         | 현재 free 좌표 F만 남긴 Hessian |
| $H_F^{-1}$ | $H_F$ 의 true inverse | Eq. 2 보상과 Eq. 3 제거에 사용   |
| F          | 아직 고정하지 않은 좌표        | 보상을 받을 수 있는 weight 집합    |
| q          | 이번에 양자화할 좌표          | 고정 뒤 되돌리지 않음             |

---

$H_F$ 는 한 layer의  $X_F$ 에서 오며 전체 Transformer Hessian이 아니다

---

# OBQ의 핵심: 하나를 고정하고 나머지를 보상

Greedy order:  $(\text{quant}(w_q) - w_q)^2 / [H_F^{-1}]_{qq}$ 가 작은 좌표부터

- |    |           |                                            |
|----|-----------|--------------------------------------------|
| 01 | 좌표 $q$ 선택 | scaled error가 작은 weight를 고른다               |
| ↓  |           |                                            |
| 02 | grid에 고정  | $w_q$ 를 $\text{quant}(w_q)$ 로 정한다          |
| ↓  |           |                                            |
| 03 | 오차 계산     | 고정 전후 차이를 $H_F^{-1}$ 대각으로 나눈다              |
| ↓  |           |                                            |
| 04 | F를 보상     | $H_F^{-1}$ 의 $q$ column 방향으로 남은 weight를 이동 |
| ↓  |           |                                            |
| 05 | 다음 좌표     | $q$ 를 제거한 inverse로 과정을 반복                  |

정확도의 원천은 독립 반올림이 아니라 순차 보상이다

# 교육용 두 weight 예시: 입력 준비

실제 모델 측정이 아니라 보상 계산을 다시 수행할 수 있게 만든 작은 예시

| 항목                      | 값                                    | 확인                   |
|-------------------------|--------------------------------------|----------------------|
| 한 row w                 | [0.80, 0.55]                         | weight 두 개           |
| grid                    | {0, 1}                               | 가장 가까운 값으로 고정        |
| 입력 X                    | [[1,0],[1,1]]                        | 두 calibration column |
| H와 true H <sup>-1</sup> | [[2,2],[2,4]], [[1,-0.5],[-0.5,0.5]] | 곱하면 identity         |

첫 좌표 0.80을 먼저 양자화하고 둘째 좌표의 변화를 추적한다

# 보상은 0.55의 최종 반올림을 바꾼다

| 단계       | 계산                                             | 결과                      |
|----------|------------------------------------------------|-------------------------|
| 첫 좌표 고정  | $q_0 = 1$ , 오차 $0.80 - 1 = -0.20$              | 첫 weight는 1             |
| 남은 좌표 보상 | $\text{delta} = -(-0.20) / 1 \times [1, -0.5]$ | 0.55 $\rightarrow$ 0.45 |
| 둘째 좌표 고정 | 0.45를 grid에 반올림                                | 최종 $[1, 0]$             |
| 출력 loss  | 보상 $[1, 0]$ : 0.425                            | 독립 $[1, 1]$ : 0.625     |

---

이 예시에서는 Hessian-aware 순차 보상이 출력 재구성 loss를 낮춘다

---

# 그대로의 OBQ는 거대 행렬에서 비싸다

## OBQ 동작

greedy 순서 선택

inverse 갱신

행렬  $W$

중간 규모 사례

## 반복 단위

각 row마다 다름

weight를 고정할 때마다

$d_{\text{row}} \times d_{\text{col}}$

ResNet-50

## 비용

row별 탐색과 상태가 필요

많은 작은 갱신이 반복

$O(d_{\text{row}} d_{\text{col}}^3)$

단일 GPU에서 약 1시간

---

GPTQ는 보상 원리를 버리지 않고 반복 구조를 재배치한다

---

# OBBQ: row마다 active column이 다르다

Q는 고정 완료, next는 이번 선택, F는 아직 free인 실제 weight 좌표다

| row | c1        | c2        | c3     | c4        |
|-----|-----------|-----------|--------|-----------|
| r1  | w11: Q    | w12: next | w13: F | w14: F    |
| r2  | w21: Q    | w22: F    | w23: F | w24: next |
| r3  | w31: next | w32: F    | w33: F | w34: F    |

---

row별 F\_r mask와  $H_{F_r}^{-1}$  진행이 달라져 하나의 column update로 공유하기 어렵다

---

# GPTQ: 같은 active column이 세로로 정렬된다

세 row가 같은 c2를 처리하므로 하나의 F mask와 working progression을 공유한다

| row | c1     | c2        | c3     | c4     |
|-----|--------|-----------|--------|--------|
| r1  | w11: Q | w12: next | w13: F | w14: F |
| r2  | w21: Q | w22: next | w23: F | w24: F |
| r3  | w31: Q | w32: next | w33: F | w34: F |

---

H\_F는 X\_F에만 의존하므로 모든 row의 progression을 d\_col번만 갱신한다

---

# 공통 순서가 바뀌는 계산 복잡도

| 방법          | 복잡도                                                          | 왜 줄어드는가                |
|-------------|--------------------------------------------------------------|------------------------|
| 원래 OBQ      | $O(d_{\text{row}} d_{\text{col}}^3)$                         | row별 inverse 진행을 반복    |
| GPTQ Step 1 | $O(\max(d_{\text{row}} d_{\text{col}}^2, d_{\text{col}}^3))$ | inverse 진행을 column별 공유 |
| 논문의 표현      | $\min(d_{\text{row}}, d_{\text{col}})$ 배                     | 해당 재구조화의 이론적 감소 계수     |

---

이 개선은 공통 column 순서에서 생기며 block 실행의 역할과 다르다

---

# Lazy block 상태: B=2 교육용 행렬

논문은 B=128; 여기서는 좌표 상태를 보이려고 c2와 c3만 한 block으로 그린다

| row | c1 past | c2 active  | c3 block    | c4 later   |
|-----|---------|------------|-------------|------------|
| r1  | w11: Q  | w12: quant | w13: update | w14: defer |
| r2  | w21: Q  | w22: quant | w23: update | w24: defer |
| r3  | w31: Q  | w32: quant | w33: update | w34: defer |

---

c3는 inner loop에서 즉시 갱신하지만 c4는 block 종료 전까지 바꾸지 않는다

---

# E buffer가 later column update를 모은다

각 e는 inner loop에서 즉시 기록되고, later update만 block 끝까지 지연된다

| row | $E(:,0)$     | $E(:,1)$     | c4 global update                       |
|-----|--------------|--------------|----------------------------------------|
| r1  | e10 from w12 | e11 from w13 | $w14 \leftarrow w14 - E1 R\_block, c4$ |
| r2  | e20 from w22 | e21 from w23 | $w24 \leftarrow w24 - E2 R\_block, c4$ |
| r3  | e30 from w32 | e31 from w33 | $w34 \leftarrow w34 - E3 R\_block, c4$ |

---

전체식:  $W(:,later) \leftarrow W(:,later) - E R(block,later)$

---

## 변경 3: damping과 Cholesky로 안정화한다

|    |                  |                                                                |
|----|------------------|----------------------------------------------------------------|
| 01 | 문제               | 반복 inverse update의 수치 오차가 누적                                   |
| ↓  |                  |                                                                |
| 02 | damped Hessian   | $H_{\text{tilde}} = 2XX^T + \lambda I$ , $\lambda$ 는 평균 대각의 1% |
| ↓  |                  |                                                                |
| 03 | true inverse     | 수치적으로 안정화한 $H_{\text{tilde}}^{-1}$ 을 계산                        |
| ↓  |                  |                                                                |
| 04 | working factor R | $R = \text{chol}(H_{\text{tilde}}^{-1})^T$ 를 미리 계산             |
| ↓  |                  |                                                                |
| 05 | 순차 사용            | column loop에서 R의 필요한 row 조각을 읽음                                |

Cholesky는 exactness 보장이 아니라 안정성과 효율을 위한 재구성이다

# Algorithm 1을 다섯 동작으로 읽기

Q는 양자화 출력, E는 scaled error buffer, R은 Cholesky working factor다

- |    |            |                                                        |
|----|------------|--------------------------------------------------------|
| 01 | R 준비       | $R = \text{chol}((2XX^T + \lambda I)^{-1})^T$ 를 계산     |
| ↓  |            |                                                        |
| 02 | block 시작   | Q와 E를 준비하고 B column 범위를 선택                             |
| ↓  |            |                                                        |
| 03 | column 양자화 | 모든 row의 $W(:,j)$ 를 $Q(:,j)$ 로 고정                       |
| ↓  |            |                                                        |
| 04 | block 보상   | E와 $R(j,j:\text{block})$ 으로 block의 남은 weight 갱신        |
| ↓  |            |                                                        |
| 05 | 뒤쪽 보상      | E $R(\text{block}, \text{later})$ 로 이후 column을 한 번에 갱신 |

하나의 layer 행렬에서 같은 column을 모든 row가 병렬로 처리한다

# Calibration은 quantized block 출력을 이어받는다

|    |           |                                  |
|----|-----------|----------------------------------|
| 01 | 표본 수집     | C4에서 2048-token segment 128개 선택  |
| ↓  |           |                                  |
| 02 | block 적재  | Transformer 6개 layer 묶음을 GPU에 올림 |
| ↓  |           |                                  |
| 03 | H 누적      | 현재 block의 실제 입력으로 layer별 X와 H 계산 |
| ↓  |           |                                  |
| 04 | block 양자화 | 각 layer의 Hessian을 누적하고 GPTQ를 적용  |
| ↓  |           |                                  |
| 05 | 다음 입력     | 양자화된 block의 출력을 다음 block 입력으로 전달 |

후반 block은 앞선 양자화 오차가 반영된 입력으로 calibration된다

# RTN과 GPTQ는 같은 grid에서 비교한다

| 항목          | RTN              | GPTQ                 |
|-------------|------------------|----------------------|
| Grid        | row별 비대칭 min-max | 동일한 row별 비대칭 min-max |
| 값 선택        | 각 weight를 독립 반올림 | 순차 양자화와 오차 보상        |
| Calibration | 값 선택에 사용하지 않음    | X에서 H와 재구성 민감도 계산    |
| 비교 목적       | 빠른 대규모 baseline  | 보상 절차의 정확도 효과 확인     |

---

주요 비교의 차이는 더 유리한 grid가 아니라 reconstruction 절차다

---

# OPT-175B에서 본 perplexity 차이

WikiText2 PPL이며 낮을수록 좋다

| 표현             | Bit | PPL   | 해석                  |
|----------------|-----|-------|---------------------|
| Full precision | 16  | 8.34  | 비압축 기준              |
| GPTQ           | 4   | 8.37  | 기준보다 0.03 증가        |
| RTN            | 4   | 10.54 | 같은 grid의 독립 반올림     |
| GPTQ           | 3   | 8.68  | 이 설정에서 낮은 증가        |
| RTN            | 3   | 7.3e3 | 이 모델과 설정에서 collapse |

한 데이터셋의 PPL 결과이며 모든 task와 모델 품질을 대표하지 않는다

# Grouping은 정확도와 metadata를 함께 바꾼다

| 설정          | OPT-175B Wiki2 | 추가 조건                       |
|-------------|----------------|-----------------------------|
| GPTQ 3-bit  | 8.68           | row 전체 grid                 |
| 3-bit g1024 | 8.45           | 1024개 연속 weight마다 grid      |
| 3-bit g128  | 8.45           | 128개마다 scale과 zero metadata |
| 2-bit g128  | 9.58           | 평균 약 2.2 bit라는 별도 계산        |
| 2-bit g32   | 8.94           | 평균 약 2.6 bit, metadata 증가   |

code bit만 비교하지 말고 group 크기와 평균 저장비용을 함께 본다

# Algorithm, packing, kernel은 서로 다른 단계다

|    |                |                                   |
|----|----------------|-----------------------------------|
| 01 | GPTQ algorithm | 재구성 오차를 보고 양자화 값을 결정              |
| ↓  |                |                                   |
| 02 | Weight code    | 선택한 grid level을 낮은 bit 정수로 표현     |
| ↓  |                |                                   |
| 03 | Packed storage | 일반적 배포에서 code와 scale, zero를 함께 배치 |
| ↓  |                |                                   |
| 04 | Runtime kernel | code를 동적으로 복원하며 vector와 곱함        |
| ↓  |                |                                   |
| 05 | Model output   | KV cache와 비양자화 연산을 포함해 token 생성   |

GPTQ의 second-order 값 선택만으로 latency가 자동 개선되지는 않는다

# Memory와 latency 수치에는 실행 조건이 붙는다

| 항목            | FP16         | GPTQ 3-bit       | 조건                                |
|---------------|--------------|------------------|-----------------------------------|
| Model memory  | 5 × A100     | 약 63GB, 1 × A100 | OPT-175B, embedding과 output은 FP16 |
| KV history    | 별도           | 약 9GB            | 최대 2048 token                     |
| A100 latency  | 230 ms/token | 71 ms/token      | 5 GPU 대 1 GPU                     |
| A6000 latency | 589 ms/token | 130 ms/token     | 8 GPU 대 2 GPU                     |
| 측정 공통         | batch 1      | batch 1          | 길이 128 생성, custom kernel          |

GPU 수와 kernel까지 다른 end-to-end 비교이므로 보편 배율이 아니다

# 속도는 memory-bound 생성에서 나온다

## 실행 형태

낮은 batch 생성

추가 compute

큰 batch

Activation

## 주된 병목

matrix-vector memory 이동

dequantization 연산

matrix-matrix compute 가능

이 논문의 병목 대상 아님

## 논문의 처리

압축 code를 읽고 동적 dequantize

읽는 memory 감소로 상쇄 가능

같은 custom kernel 결과를 일반화 못함

실험에서는 양자화하지 않음

---

논문은 memory movement를 줄이며 수학적 곱셈 횟수를 줄인다고 주장하지 않는다

---

# GPTQ가 확립한 것과 남겨 둔 경계

## 확립한 범위

OPT와 BLOOM의 weight-only PTQ

calibration X의 layer 출력 재구성

shared order, lazy block, Cholesky

조건부 custom-kernel latency

ICLR 2023 paper 결과

## 확립하지 않은 범위

activation까지 양자화한 결과

모든 unseen task와 성질의 보존

모든 순서가 같다는 이론적 보장

모든 batch와 hardware의 고정 speedup

후대의 act-order, true-sequential, LLaMA 결과

---

값 선택, 확장 구조, 저장 형식, 실행 kernel을 분리하면 GPTQ를 정확히 설명할 수 있다

---